ISTA 130 Final Study Guide

Key skills:

- **Define a function.**
- **Call a function and appropriately use the returned value.**
- **`for` and `while` loops:**
  - **Traverse by index.**
  - **Traverse a `for` loop by element.**
  - **Build a string, list, or dictionary.**
  - **Accumulate a sum.**
  - **Convert between if possible.**
- **Create and evaluate arithmetic, Boolean, and string expressions, and combinations thereof.**
- **Create conditional statements, i.e. `if-elif-else`.**
- **`print` and `input`.**
- **Files: open, close, read, write, traverse by line.**
- **Strings and lists: indexing, negative indexing, slicing, methods, traversing.**
- **Dictionaries: add key-value pairs, retrieve a value, traverse by key, traverse by key in sorted order.**
- **Classes: define your own type, magic methods init and repr, instance variables, creating an object of your class, accessing its data and methods from the variable it's stored in.**
- **Understand mutable vs. immutable.**

Key resources:

- **Worksheets and solutions, plus other example code.**
- **PowerPoint PDF's.**
- **Discussion section PDF's, exercises, and solutions.**
- **`All_about_functions.pdf`.**

The test will be 70% multiple choice, 30% code writing.

Know everything from the midterm study guide.

Know the built-in Python types we have studied and understand how to manipulate objects of those types. These types include `list`, `dict`, `str`, `int`, `float`, and `bool`. More specifics about what to know are below. `Turtle` is not on the final.

There will be an expression evaluation section. There will be questions like the quizzes, except easier.

Know how the `in` operator works on strings, lists, and dictionaries.

Know how to evaluate arithmetic expressions. These expressions use the arithmetic operators `+`, `-`, `*`, `/`, `//`, `%`, `**`. Understand the difference between integer and floating point division and that floating point division always returns a float, even if the dividend and divisor were integers. Example:

```
(num_squares + num_circles) / num_polygons
```

Know how to evaluate Boolean expressions, which always evaluate to either `True` or `False`. These expressions use the relational (comparison) operators <, >, <=, >=, ==, and !=. They can also contain arithmetic operators. Example:

```
(num_squares + num_circles) / num_polygons <= 40
```

They may also include the logical operators `and`, `or`, and `not`. Notice that the above Boolean expression doesn't contain the keyword `if` anywhere. If statements make use of Boolean expressions, however Boolean expressions can exist without having an associated if statement.

Know the `len` function.

Know the `split` method – and that it always returns a list of strings.

Know the `join` method.

Know that string literals are always enclosed in quotes, with the exception of user input (see the `input` section below).

Know that `input` always returns a string, although the user doesn't type quotes.

Know the `replace`, `count`, and `upper`/`lower` methods. What classes have you met that have `count` methods? What class do the others belong to?

Understand how the `print` function works. Know that it can take multiple arguments (separated by commas) and that these arguments are printed out separated by a single space.

Understand the functions `max`, `sum`, and `round`.

Understand how the comparison of strings using logical operators works, including when the characters are numerals.

Know the list method `sort`. What does `sort` return? Know the `sorted` function. What does the `sorted` function return when you pass it a dictionary? When you pass it a list?

Understand indexing and slicing. Understand what these expressions evaluate to:

```
'rat'[5]
'rat'[-1]
```

```
'rat'[5:]
```

Understand how to apply indexing and slicing to lists (the above examples are strings).

Understand string concatenation (addition) and multiplication.

Understand conditional execution of code – in other words `if – elif – else` statements. Be able to determine the result of code fragments containing such statements. Be able to write code containing such statements. As part of this, you will need to understand Python indentation. You will also have to be able to create Boolean expressions in Python that define conditions for execution. Know that `else` catches all remaining cases and therefore does not need a condition.

Be able to translate instructions given in English into Python.

Be able to define functions given specifications about their functionality and parameters. As part of this, you will have to understand Python indentation and know the function definition syntax:

```
def fname(param1, param2, …):
     <function body>
```

Understand and be able to write `for` loops. Example:

```
for i in range(7):
    print((i + 1)**2)
```

Understand what happens if you modify the loop control variable in the body of a `for` loop.

Know how to traverse lists and strings both by index and element using `for` loops. Understand the limitations of traversing by element.

Understand and be able to write `while` loops. Example:

```
i = 0
while i < 7:
    print((i + 1)**2)
    i += 1
```

Understand what happens if you modify the loop control variable in the body of a `while` loop.

Know how to build strings, lists, and dictionaries in loops. What is the important list method that you need to know to add an item to the end of the list? How about the beginning? How do you get rid of the last element? The element at position `i`?

Know how to create and modify lists and dictionaries. Understand that lists are indexed by integers, but dictionaries are indexed by keys, which may be any immutable type. Be able to add, remove, and access elements from both data structures.

Know how to traverse a dictionary by key using a `for` loop. Know how to do it with the keys in sorted order.

Know how to open, read (traverse by line using a `for` loop) or write to, and close a text file. **Know how to work with csv files.**